

App Dev I Project Report

1. Student Details

Name: Ashish Kushwaha

Roll Number: DS25F1000169

Email: 25f1000169@ds.study.iitm.ac.in

About Me: I am a Student at IIT Madras BS Degree program

2. Project Details

Project Title: Placement Portal Application

Problem Statement:

To design and build a web-based application that allows **Admin (Institute), Company, and students** to interact with the system based on their roles..

Approach:

Backend Logic: Developed using Python and Flask, utilizing session-based authentication to manage different access levels for Admin, Company, and Student users.

Data Persistence: Implemented via **SQLAlchemy** with an SQLite database to maintain records of Admin, Company, Student, Drive and Applications.

User Flow:

1. **Admin Oversight:** The Admin acts as the system controller, possessing the highest level of access to approve or reject new company registrations and monitor all activities.
 2. **Company:** Organizations register their profiles and, upon Admin approval, gain the ability to create and manage recruitment drives.
 3. **Student:** Registered students browse active drives from approved companies, apply for roles, and track their recruitment status (e.g., Shortlisted, Selected).
-

3. AI/LLM Declaration

I used **claude** to make model not copy pasted
The extent of AI/LLM usage is around **10-15%**, for code suggestions and making this document.
Also use to make session for authentication

4. Technologies and Frameworks Used

Technology / Library	Purpose
Flask	Core backend web framework
Flask SQLAlchemy	Object Relational Mapper for SQLite database
Jinja2	Template engine for rendering dynamic HTML pages
Bootstrap 5	Frontend styling and responsive design
CSS	For styling
SQLite	Lightweight local database for storing data
Python	Handling application timestamps and deadlines

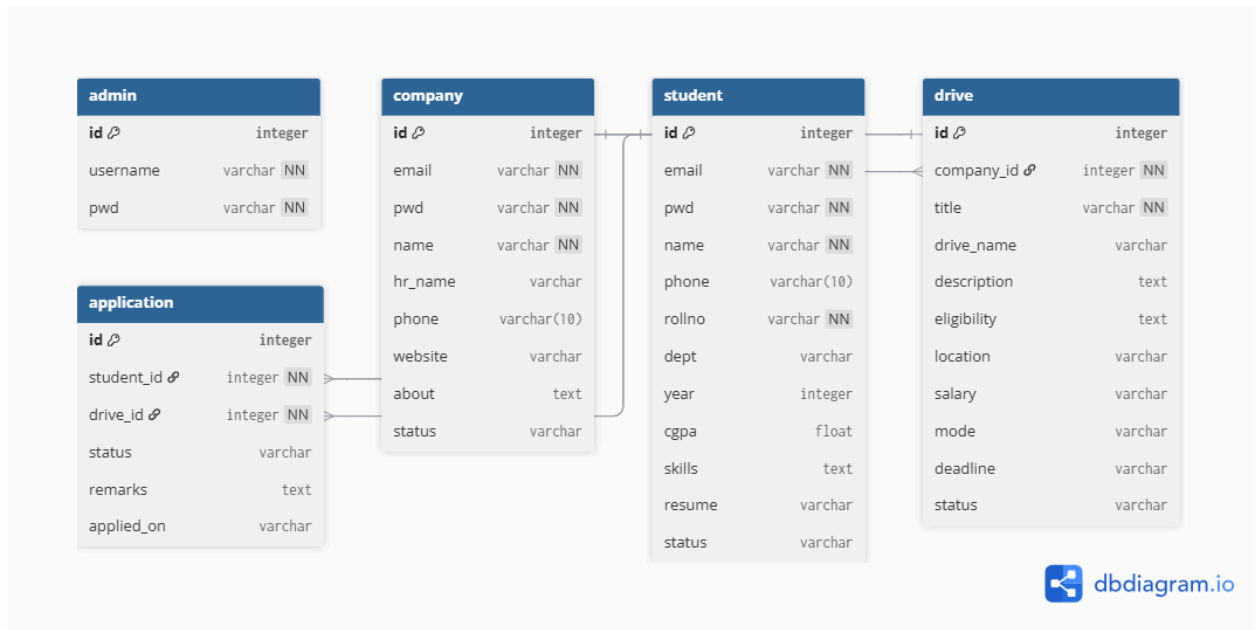
5. Database Schema / ER Diagram

Tables:

1. **Admin**— stores admin user_id and password
(id, username, pwd)
2. **Company**— Stores organization profiles and their verification status
(id, email, pwd, name, hr_name, phone, website, about, status)
3. **Student** — Stores academic records and professional details
(id, email, pwd, name, phone, rollno, dept, year, cgpa, skills, resume, status)
4. **Application**— A mapping table linking students to drives they have applied for.
(id, student_id (FK), drive_id (FK), status , remarks, applied_on)
5. **Drive**: Represents a job posting or recruitment event created by a company.
(id, company_id (FK), title, drive_name, description, eligibility, location, salary, mode, deadline, status)

Relationships:

- One-to-Many → Company → Drive
- One-to-Many → Student → Application
- One-to-Many → Drive → Application



6. API Resource Endpoints

Endpoint	Methods	Description
/login	GET / POST	Authenticates Admin, Company, or Student sessions and sets role-based session data
/logout	GET	Clears the current session and redirects to the login page.
/register/company	GET / POST	Allows new companies to submit profiles for Admin approval; checks for existing emails.
/register/student	GET / POST	Allows students to create a profile including roll number, department, CGPA, and resume.
/admin	GET	Dashboard for Admin to view statistics and manage pending/active companies and drives.

<code>/admin/search</code>	GET / POST	Enables Admin to search across all Students (by name/email/roll no) and Companies (by name/email).
<code>/approved_comp_application/<int:company_id></code>	GET	Administrative route to approve a pending company registration.
<code>/blacklist_reg_company/<int:company_id></code>	GET	Admin blocks a company and automatically sets all their associated drives to "Closed".
<code>/blacklist_student/<int:student_id></code>	GET	Administrative route to block a student from participating in recruitment activities.
<code>/approve_drive/<int:drive_id></code>	GET	Admin approves a "Pending" drive created by a company to make it "Active" for students.
<code>/company/<int:company_id></code>	GET	Dashboard for companies to view their active, closed, and pending recruitment drives.
<code>/create_drive/<int:company_id></code>	GET / POST	Allows approved companies to submit a new recruitment event (initially set to "Pending" status).
<code>/updates_on_drive/<int:drive_id></code>	GET	Allows companies to view all student applications received for a specific drive.
<code>/review_stu_app/<int:drive_id>/<int:student_id></code>	GET / POST	Allows companies to update a student's status (Shortlist, Selected, Reject) and add specific remarks.
<code>/student/<int:student_id></code>	GET	Student dashboard to browse approved organizations and track applied drive statuses.
<code>/apply_current_drive/<int:drive_id>/<int:student_id></code>	GET / POST	Creates a unique entry in the Application table; prevents duplicate applications for the same drive.
<code>/edit_student/<int:student_id></code>	GET / POST	Allows students to update their profile information, academic records, and resume links.

```
/student_history/< GET  
int:student_id>
```

Displays a comprehensive history of all drives a student has applied to with their respective results.

7. Architecture and Features (optional)

Architecture Overview:

- **app.py** – main Flask application entry point
- **application/models.py** – database models using SQLAlchemy
- **application/controllers.py** – Flask Blueprints for all and activity routes
- **/templates** – Jinja2 HTML templates
- **/static** – CSS

Implemented Features:

1. Admin: The System Controller

- **Gatekeeping:** Approves or rejects new company registrations before they can post drives.
- **User Oversight:** Can blacklist students or companies; blocking a company automatically closes all their active drives.
- **Drive Approval:** Reviews and activates "Pending" job drives created by companies.
- **Monitoring:** A dashboard showing total counts of students, companies, and applications.
- **Global Search:** Search for students and companies using names, emails, or roll numbers.

2. Company: The Recruiter

- **Drive Management:** Creates recruitment drives with details like salary, eligibility, and deadlines.
- **Application Review:** Views all students who applied for their drives.
- **Hiring Workflow:** Updates student status to "Shortlisted," "Selected," or "Rejected" with custom remarks.

3. Student: The Applicant

- **Profile Management:** Edits academic details, skills, and resume links.
- **Job Discovery:** Browse approved organizations and their active recruitment drives.
- **One-Click Apply:** Submits applications for drives while the system prevents duplicate entries.
- **History Tracking:** A personal dashboard to check application status and company feedback.

4. Technical & Security

- **Role-Based Access:** Uses session checks to ensure users only access their authorized dashboards.
- **Data Integrity:** Uses database constraints to ensure application accuracy and "Active" status requirements for login.
- **Responsive UI:** Built with Bootstrap 5 for clean navigation and dashboard layouts.

8. Video Presentation

Drive Link:

<https://drive.google.com/file/d/1bKEWPVmm-wdIPgph7PE2yLWIPZEmi1WK/view?usp=sharing>